

Вечеринка с луковыми блинами

Bohan устраивает вечеринку. Поскольку он любит луковые блины цун ю бин, он решает купить их в ближайшем магазине для своей вечеринки. В магазине продаются цун ю бин N различных вкусов, пронумерованных от 0 до $N - 1$. Bohan решает купить по N блинов каждого вкуса, всего N^2 блинов. Каждый блин разделён на K одинаковых ломтиков перед запаковкой в свой собственный пакет. Пакеты пронумерованы от 0 до $N^2 - 1$. Пусть $F[i]$ обозначает вкус блина для пакета i .

Во время вечеринки Bohan приглашает всех гостей поиграть в игру. Игра проходит следующим образом.

Сначала Bohan выбирает N гостей и приводит их в комнаты с номерами $0, 1, \dots, N - 1$ так, чтобы в каждой комнате был ровно один гость. Остальные гости остаются снаружи до конца игры. Все N комнат выглядят одинаково, поэтому гости не могут сказать, в какой комнате они находятся.

Затем для каждой комнаты i ($0 \leq i < N$) Bohan заходит в комнату i и тайно сообщает гостю целое число $P[i]$, представляющее **запрещенный в данной комнате вкус**. Он может также сообщить гостю, в какой комнате они находятся, а может и нет. Bohan гарантирует, что $[P[0], P[1], \dots, P[N - 1]]$ является перестановкой $[0, 1, \dots, N - 1]$.

Затем проводится N раундов. В i -м раунде Bohan приносит пакеты $0, 1, \dots, N^2 - 1$ один за другим в последовательном порядке в комнату $i - 1$. Когда гость в комнате $i - 1$ получает пакет j , он узнает:

- $F[j]$ — вкус блина, содержащегося в пакете j , и
- количество оставшихся ломтиков в пакете.

Прежде чем получить следующий пакет, гость в комнате $i - 1$ должен решить, сколько ломтиков съесть из текущего пакета. Количество ломтиков, которое они могут съесть, зависит от ситуации:

- Если $P[i - 1] \neq F[j]$, гость может удалить из пакета любое количество ломтиков и съесть их.
- Если $P[i - 1] = F[j]$, гостю не разрешается есть ломтики.

Обратите внимание, что гости заранее не знают последовательность $F[0], F[1], \dots, F[N^2 - 1]$.

После всех N раундов гостям снаружи выдается последовательность пакетов. Увидев пакеты, они узнают вкусы блинов и количество оставшихся ломтиков в каждом пакете. Имея эту информацию, они должны определить значения $P[0], P[1], \dots, P[N - 1]$.

Гостям разрешается общаться до начала игры. Они также заранее знают значения N и K . Ваша цель — реализовать стратегию, которая позволяет гостям снаружи всегда правильно определять значения $P[0], P[1], \dots, P[N - 1]$.

Детали реализации

Вам нужно реализовать три функции.

Вы должны реализовать следующие две функции для гостей в комнатах $0, 1, \dots, N - 1$.

```
void init(int N, int K, int p, int r)
```

Предположим, гость находится в комнате i .

- N : количество вкусов блинов.
- K : количество ломтиков каждого блина до начала игры.
- $p = P[i]$: запрещенный вкус для еды в комнате i .
- Если Bohan решает сообщить гостю, в какой комнате он находится, то $r = i$. В противном случае $r = -1$.

```
int strategy(int b, int f, int s)
```

Предположим, гость находится в комнате i .

- b : номер текущего пакета.
- $f = F[b]$: вкус блина для пакета b .
- s : количество ломтиков, оставшихся в пакете b .
- Эта функция должна возвращать неотрицательное целое число x , представляющее количество ломтиков, которые гость решает съесть.
 - Если $f \neq P[i]$, то должно выполняться $0 \leq x \leq s$.
 - Если $f = P[i]$, то должно выполняться $x = 0$.

Вы должны реализовать следующую функцию для гостей снаружи.

```
std::vector<int> guess(int N, int K, std::vector<int> F,  
                      std::vector<int> S)
```

- N : количество вкусов блинов.

- K : количество ломтиков каждого блина до начала игры.
- F : массив длины N^2 , где $F[i]$ — вкус блина в пакете i .
- S : массив длины N^2 , где $S[i]$ — количество ломтиков, оставшихся в пакете i после всех N раундов.
- Эта функция должна возвращать массив P длины N , где $P[i]$ — число, данное гостю в комнате i .

Каждый тестовый пример включает T независимых игр.

В процессе проверки для каждой из T игр будет использован $N + 1$ процесс. Для каждого i такого, что $0 \leq i < N$, процесс i представляет гостя в комнате i . Процесс N представляет гостей снаружи. Для каждого i такого, что $0 \leq i < N$, процесс $(i + 1)$ запускается после завершения процесса i .

Для процессов от 0 до $N - 1$:

- `init` вызывается ровно один раз.
- `strategy` вызывается N^2 раз после `init`.
 - Гарантируется, что для j -го вызова $b = j - 1$.

Для процесса N :

- `guess` вызывается ровно один раз.

Тестирующая система может чередовать процессы из разных игр, но гарантируется, что относительный порядок процессов в каждой отдельной игре сохраняется.

Ограничения

- $1 \leq T \leq 400$.
- $2 \leq N \leq 30$.
- Сумма N^3 по всем играм в тестовом примере не превышает 27 000.
- $1 \leq K \leq N$
- $K \in \{1, 3, N\}$.
- $[P[0], P[1], \dots, P[N - 1]]$ является перестановкой $[0, 1, \dots, N - 1]$.
- $0 \leq F[j] < N$ для каждого j такого, что $0 \leq j < N^2$.
- Каждое i встречается ровно N раз в $[F[0], F[1], \dots, F[N^2 - 1]]$ для каждого i такого, что $0 \leq i < N$.
- Грейдер **не адаптивен**, то есть $[P[0], P[1], \dots, P[N - 1]]$ и $[F[0], F[1], \dots, F[N^2 - 1]]$ фиксированы до первого вызова `init`.

Подзадачи

Подзадача	Оценка	Дополнительные ограничения
1	8	$K = 1, N = 2$.
2	7	$K = 1$ и Воһан решает сообщить каждому, в какой комнате он находится.
3	14	$K = 1$ и $[F[i \cdot N], F[i \cdot N + 1], \dots, F[i \cdot N + N - 1]]$ является перестановкой $[0, 1, \dots, N - 1]$ для каждого i такого, что $0 \leq i < N$.
4	18	$K = N$.
5	21	$K = 3, N \geq 3$.
6	32	$K = 1$.

Пример

Рассмотрим ситуацию, в которой $T = 1$ и $N = 2, K = 2, P = [1, 0], F = [1, 0, 0, 1]$ для единственной игры в тестовом примере.

Пусть $S[i]$ обозначает текущее количество оставшихся ломтиков в пакете i . Изначально $S = [2, 2, 2, 2]$.

Предположим, гости определили следующую стратегию до начала игры.

- Для гостей в комнатах, если им разрешено есть текущий блин:
 - Если это первый блин, который они могут съесть, то они съедят все оставшиеся ломтики.
 - В противном случае они съедят только один ломтик.
- Для гостей снаружи:
 - Если $S = [0, 0, 1, 1]$, то они угадают, что $P = [1, 0]$.
 - В противном случае они угадают, что $P = [0, 1]$.

Игра проходит следующим образом.

Сначала система запускает процесс 0, представляющий гостя в комнате 0, и вызывает `init(2, 2, 1, -1)`. После инициализации грейдер делает следующие вызовы:

Вызов функции	Возвращаемое значение
<code>strategy(0, 1, 2)</code>	0
<code>strategy(1, 0, 2)</code>	2
<code>strategy(2, 0, 2)</code>	1
<code>strategy(3, 1, 2)</code>	0

Первый и четвертый вызовы должны вернуть 0, поскольку $P[0] = F[0] = F[3] = 1$.

После завершения этого процесса (заканчивается раунд 1) $S = [2, 0, 1, 2]$.

Затем система запускает процесс 1, представляющий гостя 1, и вызывает `init(2, 2, 0, -1)`. После инициализации грейдер делает следующие вызовы:

Вызов функции	Возвращаемое значение
<code>strategy(0, 1, 2)</code>	2
<code>strategy(1, 0, 0)</code>	0
<code>strategy(2, 0, 1)</code>	0
<code>strategy(3, 1, 2)</code>	1

Обратите внимание, что второй и третий вызовы должны вернуть 0, поскольку $P[1] = F[1] = F[2] = 0$.

После завершения этого процесса (заканчивается раунд 2) $S = [0, 0, 1, 1]$.

Наконец, система запускает процесс 2, представляющий гостей снаружи. Грейдер делает следующие вызовы:

Вызов функции	Возвращаемое значение
<code>guess(2, 2, [1, 0, 0, 1], [0, 0, 1, 1])</code>	<code>[1, 0]</code>

Гости снаружи угадали правильную перестановку. Таким образом, тестовый пример считается правильным.

Обратите внимание, что этот подход не всегда позволяет гостям снаружи правильно угадать перестановку.

Аттачмент к этой задаче содержит другой пример ввода в дополнение к этому примеру.

Грейдер для участника

Грейдер для участника поддерживает **только одну игру на тестовый пример**.

Формат ввода:

```
N K
P[0] P[1] ... P[N-1]
F[0] F[1] ... F[N*N-1]
reveal
```

- `reveal` равно либо 0, либо 1.
 - Если `reveal` равно 0, грейдер для участника будет действовать так, как будто Bohan никому не сообщает номер их комнаты.
 - Если `reveal` равно 1, грейдер для участника будет действовать так, как будто Bohan сообщает каждому номер их комнаты.

Если массив, возвращенный `guess`, совпадает с $[P[0], P[1], \dots, P[N - 1]]$, грейдер для участника выводит

```
Accepted
```

Кроме того, грейдер для участника выводит вызовы функций в порядке их выполнения для целей отладки.